Lab Handout 1-C
Serial Peripheral Interface (SPI) Bring-Up: STM32 ↔ SX1276 Register Access

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

1 Objective

Establish reliable SPI communication between the STM32 Nucleo-L476RG and the SX1276-based RFM95W module. You will read the `RegVersion` register and receive the expected value `0x12`, then demonstrate write/read-back and radio mode transitions. This handout is the first point at which firmware actually touches the RF silicon.

2 Prerequisites

- Lab Handout 1-A complete (breakout physically assembled).
- Lab Handout 1-B complete (toolchain working, blinky verified).

3 Materials

- Assembled RFM95W breakout (from LH1-A).
- Nucleo-L476RG + USB cable (from LH1-B).
- Half-size breadboard, 8–10 male–female jumper wires.
- Logic analyzer (optional but strongly recommended for troubleshooting).

4 Wiring

RFM95W Pin	Function	Nucleo Pin	Board Label
`VIN`	+3.3 V supply	`3V3`	3V3
`GND`	Ground	`GND`	GND
`SCK`	SPI clock	`PA5`	SCK/D13
`MISO`	SPI data in (to Microcontroller Unit (MCU))	`PA6`	MISO/D12
`MOSI`	SPI data out (from MCU)	`PA7`	PWM/MOSI/D11
`CS`	Chip select (active low)	`PB6`	PWM/CS/D10
`RST`	Reset (active low)	`PA9`	D8
`G0` (`DIO0`)	Packet-done interrupt	`PA8`	D7

Board Label refers to the silkscreen text on the Nucleo CN10 connector. Locate pins by this label; the STM32 pin name (PA5, etc.) is the CubeMX reference.

[Figure placeholder]

`LH1C_wiring_diagram.png`

Schematic of the Nucleo-L476RG ↔ RFM95W wiring. Power, ground, full-duplex SPI1, chip-select, reset, and `DIO0` interrupt line are all shown.

Figure 1: Schematic of the Nucleo-L476RG ↔ RFM95W wiring. Power, ground, full-duplex SPI1, chip-select, reset, and `DIO0` interrupt line are all shown.

Wire Before You Power

Double-check `VIN` vs. `GND`. The RFM95W will tolerate 3.3–5 V on `VIN` (it has an on-board regulator), but swapping `VIN` and `GND` *will* destroy the module. Verify with a DMM before plugging the Nucleo into USB.

[Figure placeholder]

`LH1C_breadboard_photo.jpg`

Physical breadboard build matching the schematic above. Keep jumper leads short to minimize SPI ringing.

Figure 2: Physical breadboard build matching the schematic above. Keep jumper leads short to minimize SPI ringing.

5 Procedure

Create All Projects Now — Do Not Copy or Move Folders Later

The STM32CubeMX + VS Code toolchain is sensitive to project folder location. Once a project is generated, *moving or copying its folder may break the CMake cache, the launch.json paths, and frequently the STM32 extension's project detection.* You will spend more time troubleshooting than writing firmware. The fix is to create every project you will ever need *up front*, in its final location, and never move them. This handout creates all three working projects in Step 1 before any RF code is written. Subsequent handouts (LH1-D, LH1-F, LH1-G) only *open* the appropriate pre-existing project and add code; no project copying or renaming.

5.1 Step 1 — Create all three working projects with identical baseline configuration

You will create three STM32CubeMX projects, one per Nucleo board, each with the same baseline peripheral configuration. Each project lives permanently in its own folder for the rest of Module 1 and beyond.

Table 1: Three projects created in this step, with their permanent roles.

Project Name	Used in Lab Handouts	Permanent Role
`Nucleo_RFM95W_TargetA`	LH1-C (this), LH1-D, LH1-F	DATA initiator (Beacon ID `0x01`)
`Nucleo_RFM95W_TargetB`	LH1-D, LH1-F	ACK responder (Beacon ID `0x02`)
`Nucleo_RFM95W_Threat`	LH1-G, LH1-H	Interceptor + PAUSE TX (Beacon ID `0xFF`)

Step 1.1 — Choose a permanent project root

Pick one folder on your local disk that will hold all three project folders permanently. Recommended locations:

- Windows: `C:\STM32_Capstone\` (NOT inside OneDrive, NOT in your Documents folder if Documents is OneDrive-synced).
- macOS / Linux: `~/STM32_Capstone/` (NOT in iCloud Drive, NOT in Dropbox).

Do Not Use Cloud-Synced Folders

OneDrive, iCloud Drive, Dropbox, and Google Drive will silently corrupt CubeMX-generated files mid-write because they sync the directory while the toolchain is still creating it. Symptoms include missing `.cmake` files, garbled `main.c` regenerations, and unrecoverable Vivado-style "project version mismatch" errors. Use a local-only folder.

Step 1.2 — Create `Nucleo_RFM95W_TargetA`

1. In VS Code, click the **STM32** icon in the Activity Bar, then click **Launch STM32CubeMX**.
2. In CubeMX, click **Access to Board Selector**, type `NUCLEO-L476RG` in the Commercial Part Number search box, select the board, then click **Start Project**. When prompted "Initialize all peripherals with their default Mode?" click **No** (you will configure peripherals manually).
3. Click the **Project Manager** tab.
   - Project Name: `Nucleo_RFM95W_TargetA`
   - Project Location: the permanent root from Step 1.1.
   - Toolchain / IDE: **CMake**.
4. Do **not** click Generate Code yet. Configure the peripherals as described in Steps 1.3–1.7 below, then generate code at the end.

Step 1.3 — Enable SPI1

1. In the pinout view, click pin `PA5` and select **SPI1_SCK** from the popup. Repeat for `PA6` → **SPI1_MISO** and `PA7` → **SPI1_MOSI**.
2. In the left-hand **Categories** tree, expand **Connectivity** and click **SPI1**.
3. Set the **Mode** dropdown to **Full-Duplex Master**. The pins assigned above will be confirmed in the pinout view.
4. Click the **Configuration** tab (or **Parameter Settings** in newer CubeMX versions). Set each parameter per Table 2 below. Pay particular attention to CPOL=0 and CPHA=1 Edge — an incorrect clock phase is the most common cause of a failed `RegVersion` read in Step 5. Set **NSS** to **Software** — chip select will be driven manually in firmware.

Table 2: SPI1 configuration for RFM95W register access.

Parameter	Value
Mode	Full-Duplex Master
Data size	8 bits
First bit	MSB first
Prescaler	**32** (yields 2.5 MHz at 80 MHz PCLK2; safe margin below the 10 MHz max)
CPOL	0 (Low)
CPHA	1 Edge
NSS	Software

Step 1.4 — Configure GPIO pins for CS and RST, and configure DIO0 as EXTI input

1. In the pinout view, click pin `PB6`. Select **GPIO_Output** from the popup.
2. Click pin `PA9`. Select **GPIO_Output**.
3. Click pin `PA8`. Select **GPIO_EXTI8** (NOT GPIO_Output — DIO0 is the SX1276's interrupt line, used by LH1-F's RxDone ISR and by LH1-G's Threat ISR; configuring it as EXTI now avoids a mid-project CubeMX edit later).
4. To assign user labels, click **System Core** → **GPIO** in the categories tree. Select each pin and set its **User Label**:
   - `PB6` → `RFM_CS`
   - `PA9` → `RFM_RST`
   - `PA8` → `RFM_DIO0` — in the GPIO subpanel for PA8, also set **GPIO mode** to **External Interrupt Mode with Rising edge trigger detection**.

   These labels become the `_GPIO_Port` and `_Pin` macro names in generated code (e.g., `RFM_CS_GPIO_Port`, `RFM_CS_Pin`).
5. In CubeMX **NVIC Settings** (under System Core → NVIC), enable the **EXTI line[9:5] interrupts**. This wires PA8's rising edge to the `HAL_GPIO_EXTI_Callback` that LH1-F and LH1-G install.

Step 1.5 — Enable USART2 for `printf` debug

1. In the categories tree, expand **Connectivity** and click **USART2**.
2. Set **Mode** to **Asynchronous**.
3. In **Parameter Settings**, confirm **Baud Rate** is **115200**, **Word Length** 8 bits, **Parity** None, **Stop Bits** 1.

Step 1.6 — Enable USART3 for the FPGA forwarding link (Threat use only, but configure on all three projects to keep them identical)

1. In the pinout view, click pin `PC4` and select **USART3_TX**. Click pin `PC5` and select **USART3_RX**.
2. In the categories tree, click **USART3**. Set **Mode** to **Asynchronous**. Confirm **Baud Rate** is **115200**, **Word Length** 8 bits, **Parity** None, **Stop Bits** 1.
3. In the **DMA Settings** sub-tab, click **Add**, then select **USART3_TX** from the dropdown. Leave the default settings (Memory to Peripheral, Normal mode, Byte width). LH1-G uses USART3 TX in DMA mode to forward 43-byte frames to the FPGA without blocking the main loop.

Step 1.7 — Enable TIM2 as a 1 ms tick

1. In the categories tree, expand **Timers** and click **TIM2**.
2. Set **Clock Source** to **Internal Clock**.
3. In **Parameter Settings**, set **Prescaler** = `79` and **Counter Period** = `999`. (At APB1 timer clock = 80 MHz, this produces a 1 ms periodic interrupt: $80\,\text{MHz}/(79+1)/(999+1) = 1\,\text{kHz}$.)
4. In **NVIC Settings**, enable the **TIM2 global interrupt**. LH1-F's FSM uses this 1 ms tick to maintain `ms_count` for transmit scheduling.

Step 1.8 — Generate code for `TargetA`, then bring up the project in VS Code

1. Click **Generate Code**. CubeMX will create the project in the location you specified in Step 1.2.
2. In VS Code, open the `Nucleo_RFM95W_TargetA` folder (**File → Open Folder**). The folder must be the root of the VS Code window or the STM32 extension will not detect the project.
3. Open the integrated terminal and run:

   ```
   cube bundle init --project
   cube bundle show --project
   ```

   The second command should list the board (NUCLEO-L476RG), the toolchain (CMake), and pack versions without errors.
4. Click **Setup STM32Cube project(s)** in the STM32Cube extension panel, then **Save and Close**.
5. Switch to the **Run and Debug** panel. Click **create a launch.json file** and select **STM32Cube: Launch ST-Link GDB Server**.
6. Press `F7` to build. Confirm zero errors. The project is now ready to receive code in Step 2 below.

Step 1.9 — Repeat Steps 1.2 through 1.8 for `Nucleo_RFM95W_TargetB` and `Nucleo_RFM95W_Threat`

Both projects use the *identical* peripheral configuration (SPI1, GPIO labels, EXTI, USART2, USART3 with DMA, TIM2). Only the project name and folder differ.

Why Configure USART3 + DMA + EXTI on TargetA and TargetB Even Though Only Threat Uses Them?

Configuring all three projects identically means the projects are interchangeable at the hardware level — a Target Nucleo can be flashed as a Threat and vice versa without re-running CubeMX. This costs nothing in code size (the unused peripheral init runs once at startup, $\sim$50 μs, and the unused peripherals consume no power if not started). The pedagogical benefit is large: students learn that role differentiation lives in firmware, not in hardware configuration. It also means that if a Nucleo physically fails, any other Nucleo can be substituted by reflashing the appropriate role's firmware.

After Step 1.9, you should have three folders on disk:

- `<root>/Nucleo_RFM95W_TargetA/`
- `<root>/Nucleo_RFM95W_TargetB/`
- `<root>/Nucleo_RFM95W_Threat/`

Each builds cleanly under `F7`. None will be copied, renamed, or moved for the rest of Module 1.

5.2 Step 2 — Open `TargetA` for SPI bring-up

The SPI bring-up code you write here will be removed at the start of LH1-D and replaced with the ping-pong implementation; Open `Nucleo_RFM95W_TargetA` in VS Code (if not already open) and continue.

5.3 Step 3 — Redirect `printf` to USART2

1. In `main.c`, locate the `/* USER CODE BEGIN Includes */` section near the top of the file and add:

   ```
   /* USER CODE BEGIN Includes */
   #include <stdio.h>
   /* USER CODE END Includes */
   ```

2. Locate the `/* USER CODE BEGIN PFP */` section and add prototypes for every function you will define in this lab:

   ```
   /* USER CODE BEGIN PFP */
   int __io_putchar(int ch);
   uint8_t rfm_read_reg(uint8_t addr);
   void rfm_write_reg(uint8_t addr, uint8_t val);
   uint32_t rfm_frf_to_hz(uint8_t msb, uint8_t mid, uint8_t lsb);
   /* USER CODE END PFP */
   ```

3. Scroll to the `/* USER CODE BEGIN 4 */` section near the bottom of the file and add the definition:

   ```
   /* USER CODE BEGIN 4 */
   int __io_putchar(int ch) {
       HAL_UART_Transmit(&huart2, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
       return ch;
   }
   /* USER CODE END 4 */
   ```

   Keeping the prototype and definition inside the `USER CODE` comment blocks ensures they are preserved if you regenerate code from CubeMX.

5.4 Step 4 — Write a minimal register-read function

Implement the following helper:

```
uint8_t rfm_read_reg(uint8_t addr) {
    uint8_t tx[2] = { addr & 0x7F, 0x00 };  // bit 7 = 0 => read
    uint8_t rx[2] = { 0, 0 };
    HAL_GPIO_WritePin(RFM_CS_GPIO_Port, RFM_CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_TransmitReceive(&hspi1, tx, rx, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(RFM_CS_GPIO_Port, RFM_CS_Pin, GPIO_PIN_SET);
    return rx[1];
}
```

Add the write helper immediately after in the same `USER CODE BEGIN 4` section:

```
void rfm_write_reg(uint8_t addr, uint8_t val) {
    uint8_t tx[2] = { addr | 0x80, val };  // bit 7 = 1 => write
    HAL_GPIO_WritePin(RFM_CS_GPIO_Port, RFM_CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, tx, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(RFM_CS_GPIO_Port, RFM_CS_Pin, GPIO_PIN_SET);
}

uint32_t rfm_frf_to_hz(uint8_t msb, uint8_t mid, uint8_t lsb) {
 uint32_t frf = ((uint32_t)msb << 16) | ((uint32_t)mid << 8) | lsb;
 return (uint32_t)(((uint64_t)frf * 32000000ULL) / 524288ULL);
}
```

5.5 Step 5 — Reset the module and read `RegVersion`

1. In `main.c`, locate the `/* USER CODE BEGIN 2 */` section (after all HAL and peripheral initialization). Add the following reset sequence and register read:

```
/* USER CODE BEGIN 2 */

/* Reset the RFM95W */
HAL_GPIO_WritePin(RFM_RST_GPIO_Port, RFM_RST_Pin, GPIO_PIN_RESET);
HAL_Delay(1);  // hold reset low for at least 100 us
HAL_GPIO_WritePin(RFM_RST_GPIO_Port, RFM_RST_Pin, GPIO_PIN_SET);
HAL_Delay(10); // wait for module to come out of reset

/* Read and print the version register */
uint8_t version = rfm_read_reg(0x42);
printf("RegVersion = 0x%02X\r\n", version);

/* USER CODE END 2 */
```

2. Press `F7` to build (or Command Palette → **CMake: Build**). Confirm zero errors. Use `F7` any time you want to check your code for errors without flashing.

3. Flash and run: in the **Run and Debug** panel, click the green **Start Debugging** arrow. Once the debugger halts at `main`, click the green **Continue** button in the debug toolbar.

4. Open the Serial Monitor panel:

   (a) Command Palette (`Ctrl+Shift+P`) → **Serial Monitor: Focus on Serial Monitor View**.

   (b) In the Serial Monitor panel at the bottom of VS Code, click **Open Serial Monitor**.

   (c) Select the **STLink Virtual COM Port** from the port dropdown.

   (d) Set the baud rate to **115200**.

   (e) Click **Start Monitoring**.

5. You should see the following output:

```
RegVersion = 0x12
```

   Any other value indicates a wiring or configuration error — see the Troubleshooting section.

Why 0x12?

The SX1276 silicon revision register returns `0x12` for the V1b mask set used in the RFM9x breakouts. This is the single most reliable sanity check for SPI wiring, clock polarity, and reset sequencing: if you read `0x00` or `0xFF`, the module is not talking to you; if you read anything else, you have a clock phase or byte-order issue.

[Figure placeholder]

`LH1C_regversion_printf.png`

Serial monitor output showing the first successful `RegVersion = 0x12` read. This is the canonical sanity check for the entire SPI stack.

Figure 3: Serial monitor output showing the first successful `RegVersion = 0x12` read. This is the canonical sanity check for the entire SPI stack.

5.6 Step 6 — Register write / read-back

Add the following code in `main.c` inside the `/* USER CODE BEGIN 2 */` block, immediately after the `RegVersion` read from Step 5:

1. First put the module into `SLEEP` mode — the SX1276 requires this before frequency registers can be written:

```
rfm_write_reg(0x01, 0x80);  // LoRa mode, SLEEP
HAL_Delay(10);
```

2. Write the three frequency registers to set the carrier to 906.5 MHz:

```
rfm_write_reg(0x06, 0xE2);  // RegFrfMsb
rfm_write_reg(0x07, 0xA0);  // RegFrfMid
rfm_write_reg(0x08, 0x00);  // RegFrfLsb
```

3. Read the same registers back and print each value:

```
uint8_t frf_msb = rfm_read_reg(0x06);
uint8_t frf_mid = rfm_read_reg(0x07);
uint8_t frf_lsb = rfm_read_reg(0x08);

printf("FrfMsb = 0x%02X\r\n", frf_msb);
printf("FrfMid = 0x%02X\r\n", frf_mid);
printf("FrfLsb = 0x%02X\r\n", frf_lsb);
uint32_t carrier_hz = rfm_frf_to_hz(frf_msb, frf_mid, frf_lsb);
printf("Carrier = %lu.%03lu MHz\r\n", carrier_hz /
 1000000UL, (carrier_hz % 1000000UL) / 1000UL);
```

4. Press `F7` to build, then flash using the green **Start Debugging** arrow in the **Run and Debug** panel. Check the serial monitor. The three printed values should match `0xE2`, `0xA0`, `0x00`, and the carrier print should read `Carrier = 906.500 MHz`. A mismatch usually indicates a CPOL/CPHA error or a floating `MISO` line.

How to calculate `RegFrf*`

For the SX1276 in LoRa mode, the carrier frequency is set by a 24-bit value:

$$f_{\mathrm{RF}} = \mathrm{FRF} \cdot \frac{F_{\mathrm{XOSC}}}{2^{19}}, \qquad \mathrm{FRF} = \left\lfloor \frac{f_{\mathrm{RF}}\, 2^{19}}{F_{\mathrm{XOSC}}} \right\rceil$$

Using $F_{\mathrm{XOSC}} = 32\,\mathrm{MHz}$ and $f_{\mathrm{RF}} = 906.5\,\mathrm{MHz}$:

$$\mathrm{FRF} = \left\lfloor \frac{906.5 \times 10^6 \cdot 2^{19}}{32 \times 10^6} \right\rceil = 14{,}852{,}096 = \texttt{0xE2A000}$$

So write `RegFrfMsb=0xE2`, `RegFrfMid=0xA0`, and `RegFrfLsb=0x00`.
The helper `rfm_frf_to_hz` performs this inverse in integer arithmetic using `uint64_t` to avoid overflow:

```
uint32_t rfm_frf_to_hz(uint8_t msb, uint8_t mid, uint8_t lsb) {
    uint32_t frf = ((uint32_t)msb << 16) | ((uint32_t)mid << 8) | lsb;
    return (uint32_t)(((uint64_t)frf * 32000000ULL) / 524288ULL);
}
```

The result is printed using integer formatting to avoid `%f` float-support issues in the nano C library:

```
printf("Carrier = %lu.%03lu MHz\r\n",
       carrier_hz / 1000000UL,
       (carrier_hz % 1000000UL) / 1000UL);
```

5.7 Step 7 — Mode transitions

The `RegOpMode` register (`0x01`) controls the radio operating state. The low 3 bits select the mode; bit 7 selects LoRa vs. FSK. Add the following to `/* USER CODE BEGIN 2 */` after the Step 6 code, then press `F7` to build and use the green **Start Debugging** arrow to flash. Observe each transition in the serial monitor.

1. Read the current mode and print it:

   ```
   printf("OpMode = 0x%02X\r\n", rfm_read_reg(0x01));
   ```

2. Transition to LoRa `SLEEP` (bits [2:0] = `000`):

   ```
   rfm_write_reg(0x01, 0x80);
   HAL_Delay(10);
   printf("OpMode (SLEEP) = 0x%02X\r\n", rfm_read_reg(0x01));
   ```

3. Transition to LoRa `STANDBY` (bits [2:0] = `001`):

   ```
   rfm_write_reg(0x01, 0x81);
   HAL_Delay(10);
   printf("OpMode (STANDBY) = 0x%02X\r\n", rfm_read_reg(0x01));
   ```

4. Transition to LoRa `TX` (bits [2:0] = `011`):

```
rfm_write_reg(0x01, 0x83);
HAL_Delay(10);
printf("OpMode (TX) = 0x%02X\r\n", rfm_read_reg(0x01));
```

   The module will return to `STANDBY` automatically after any transmission attempt; seeing `0x81` on a subsequent read is expected.

6 Verification Checklist

- ☐ `RegVersion` reads `0x12`.
- ☐ Write/read-back to `RegFrf*` registers returns identical values.
- ☐ `RegOpMode` mode bits transition correctly through `SLEEP`, `STANDBY`, `TX`.
- ☐ Serial output shows all register values in hex over the ST-LINK virtual COM port.

Troubleshooting

- `RegVersion` **reads** `0x00`: `MISO` not connected, or `CS` never asserted.
- `RegVersion` **reads** `0xFF`: `MISO` floating (pull-up). Check the jumper to `PA6`.
- **Reads a seemingly-random but repeatable value**: CPOL/CPHA wrong, or bit 7 (read/write) flipped in your address byte.
- **First read is wrong, subsequent reads correct**: missing reset delay; increase the post-reset wait to 20 ms.
- `Carrier =` **shows no number or garbage**: the nano C library used by default does not support `%f`. Use `rfm_frf_to_hz` with the `%lu.%03lu` integer split pattern shown in Step 6.

Lab Handout 1-C Deliverable

1. Source code for your `rfm_read_reg` / `rfm_write_reg` helpers and the `main` test sequence.
2. Serial monitor log showing `RegVersion = 0x12`, frequency register write/read-back, and all three mode transitions.
3. If a logic analyzer was used: one screenshot of a successful `RegVersion` read transaction with `CS`, `SCK`, `MOSI`, `MISO` traces.

Glossary

MCU	Microcontroller Unit	2
RF	Radio Frequency	1
SPI	Serial Peripheral Interface	1, 2, 3, 5, 10, 11